Solution Synthesis

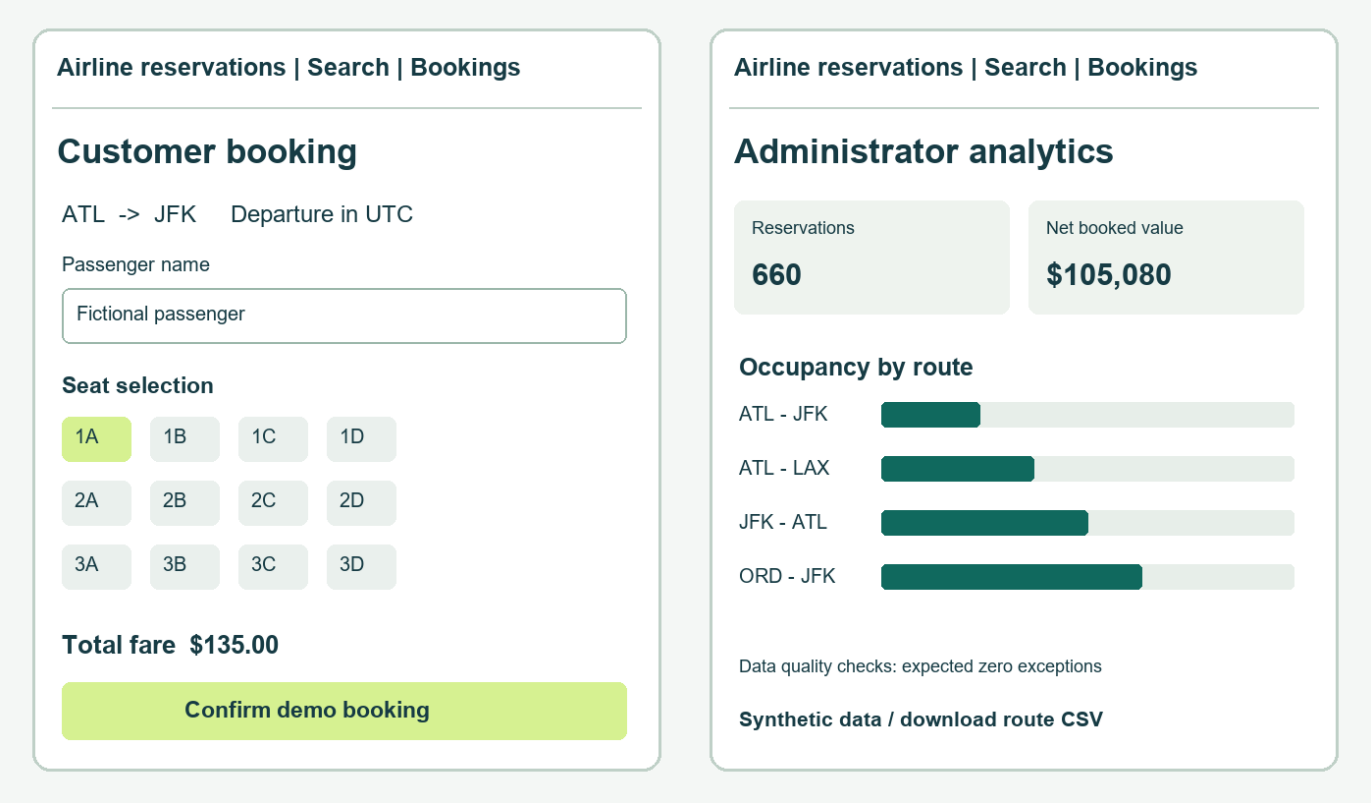
Airline Reservation System connects fourteen individually supplied HTML pages to JSON routes in Node.js. Shared app.js handles forms and renders API results; styles.css provides consistent navigation, spacing, keyboard focus and mobile breakpoints. The interface distinguishes fictional reservations from valid airline tickets.

HTML file	Access	Purpose
index.html	Public	Route/date search
register.html	Public	Customer account creation
login.html	Public	Authentication
flights.html	Public	Search results
flight-details.html	Public	Schedule, fare, aircraft, seats
booking.html	Customer	Passenger names and seat selection
confirmation.html	Customer	Reference, fare, passengers, print
my-bookings.html	Customer	Owned lookup and cancellation
admin.html	Administrator	Operations overview
admin-airports.html	Administrator	Create/edit airport details
admin-aircraft.html	Administrator	Create aircraft/seats; edit model
admin-flights.html	Administrator	Define routes, schedule, fare, cancel
admin-reservations.html	Administrator	Inspect/cancel reservations
analytics.html	Administrator	KPIs, route occupancy, CSV

Public page shells contain no protected records. Protected API routes check the session and role before returning data. Client redirects help navigation but are not the security boundary.

Interface Wireframes

These wireframes illustrate the intended booking and analytics layouts. They are design illustrations, not screenshots or proof of browser execution.



Customer journey

Search selects origin, destination and UTC date. Results show a flight number, fare and remaining seats. Details show departure, arrival, aircraft and the single-flight scope. Booking collects one to four names and a seat per passenger. The confirmation supplies the saved reference and total. My bookings lists owned reservations, supports reference filtering and asks for confirmation before cancellation.

Administrator journey

Create an airport, add an aircraft and generated seat layout, define a route, then schedule a dated departure. The server rejects an overlapping aircraft schedule. An administrator can change the selling fare or cancel a departure; cancellation updates reservations, allocations and payments together. Historical fares remain unchanged. Route definitions with instances cannot be rewritten.

Inputs Outputs and Interaction States

Surface	Validation and output
Registration	Name 1–80 characters; email syntax and max 200; password 12–128. Duplicate emails return an error.
Login	Generic invalid-credentials message. Database session lasts 8 hours. Sign-out revokes the session.
Search	Different three-letter airports and valid calendar date. No matches renders an empty-state message.
Booking	1–4 nonblank passenger names; different seats on the selected aircraft. Occupied seats disabled in UI; server rechecks.
Confirmation	Reference, status, route, date, passenger names, seats and saved total. Print stylesheet hides controls.
Administration	Required fields; valid IANA timezone; future ordered timestamps; positive fare; overlap validation.
Errors and loading	Status region announces errors. Loading placeholders appear while fetching; booking button disables during submission.

Accessibility and responsive behavior

Forms use visible labels, native controls and keyboard focus outlines. A skip link bypasses navigation. Status messages use an aria-live region. At narrow widths the search and content grids stack. Tables scroll horizontally. Occupied seat radios are disabled and labeled; status labels also use text. The owner reported successful narrow-window and keyboard checks on October 10, 2026. Print preview was visually verified after the CLI fix. Actual mobile-device and assistive-technology testing remain unverified; no accessibility compliance is claimed.

Communication and storage

The browser sends JSON using fetch. The server validates inputs, resolves the database session, checks customer ownership or administrator role, and executes parameterized MySQL statements. Mutating authenticated routes require a CSRF token. The server returns JSON and appropriate HTTP codes; no database credentials are included in HTML or JavaScript.

Feature Traceability

Feature	Page and route	Tables
Accounts	register/login; POST /api/register, /login, /logout	users, sessions
Search/details	index, flights, flight-details; GET /api/flights and /:id	airports, flights, flight_instances, aircraft, seats, seat_assignments
Group booking	booking; POST /api/bookings	bookings, passengers, booking_passengers, seat_assignments, payments
Lookup/print	confirmation, my-bookings; GET /api/bookings	bookings plus route, passenger and seat joins
Cancellation	my-bookings; POST /api/bookings/:id/cancel	bookings, seat_assignments, payments
Airport/aircraft	admin-airports/aircraft; /api/admin/airports and /aircraft	airports, aircraft, seats
Flight scheduling	admin-flights; /api/admin/flights and /instances	flights, flight_instances
Admin reservation	admin-reservations; GET /api/admin/reservations; POST /:id/cancel	bookings, users, flight_instances, flights, allocations, payments
Analytics/export	analytics; GET /api/admin/analytics and /analytics.csv	bookings, flights, flight_instances, seats, allocations, payments

Design exclusions

No connecting itineraries, discount programs, vacation planning, hotel/car inventory, email delivery or real payment gateway. Single-segment bookings keep the relationship model explainable and allow strong tests of the required workflow. Additional portfolio value comes from SQL analysis, explicit metrics and data-quality checks.